

PRATIMA VERMA

Agra, Uttar Pradesh, India | pratimaverma711@gmail.com | +91 7817806908
GitHub: github.com/pratimaverma-me | LinkedIn: linkedin.com/in/pratimaverma10

TECHNICAL SKILLS

- **Programming Languages:** Rust, C, C++, Python, SQL
- **Software Engineering:** OOP, Data Structures & Algorithms, Design Patterns, System Design, Software Architecture
- **Rust & Systems Programming:** Ownership, Borrowing, Lifetimes, Memory Safety, Error Handling, Traits, Generics, Multithreading, Concurrency
- **Operating Systems:** Linux, Process Management, Memory Management, File Systems, Shell Environment
- **Financial Systems:** Low-Latency Trading Systems, Order Book Engine, Market Data Feeds, NSE Feed Processing, MCX Feed Processing, Trade Processing
- **Performance Engineering:** Benchmarking, Profiling, Optimization, Throughput Analysis, Latency Reduction
- **Testing & Debugging:** Unit Testing, Integration Testing, GDB, LLDB, Valgrind, Logging
- **Tools & Frameworks:** PyO3, Git, Linux Server, CUDA, TensorFlow, PyTorch

EXPERIENCE

Rust Developer Intern — Raghunandan Money *Nov 2025 – Present*

Agra, Uttar Pradesh

- Developing a high-performance Order Book and Trade Processing Engine in Rust for low-latency financial market infrastructure.
- Designed and developed OrderPulse, a Rust-powered market-data processing library exposed to Python through PyO3.
- Building production-grade binary feed parsers for exchange order and trade messages with real-time decoding and event processing.
- Working on MCX and NSE market-data systems involving binary feed reading, event extraction, automation, and high-throughput pipelines.
- Implemented order book workflows supporting New, Modify, Cancel, and Trade events.
- Optimized internal storage by replacing tree-based implementations with cache-friendly array-indexed designs to reduce latency.
- Applied Rust ownership, borrowing, and concurrency models for thread-safe and memory-safe execution.
- Developed unit tests, debugging workflows, benchmarks, and profiling tools to improve reliability and performance.
- Built a high-performance Rust order book engine that processed 3.01 million filtered order and trade messages in 93.73 ms of core processing time.
- Achieved ~31.1 ns average core processing latency per message and ~32.12 million messages per second measured core throughput in release-mode benchmarks.
- Designed a binary market-data processing pipeline with separate stages for file loading, token filtering, order book manager initialization, and core low-latency message processing.

Full Stack Developer Intern — Infotech, Noida *Jun 2023 – Aug 2023*

- Developed responsive web applications using HTML, CSS, JavaScript, and Bootstrap.
- Integrated ASP.NET services with SQL databases for secure backend operations and data management.
- Participated in debugging, testing, and deployment activities across development environments.

PROJECTS UNDERTAKEN

OrderPulse – Rust Market Data Processing Library 2026

High-performance Rust library with Python bindings for processing NSE and MCX binary market-data feeds, constructing real-time order books, and exposing low-latency market data to Python applications. PyPI: pypi.org/project/orderpulse

- Designed and published a developer-focused market-data library enabling Python applications to use Rust-level performance.
- Implemented MessageCacheReader for full-file memory caching and StreamingBinaryLoader for large binary feed files.
- Developed OrderbookBuilder for market depth, best bid/ask, spread, mid-price, and real-time order book snapshots.
- Built Python APIs using PyO3, exposing decoded order and trade messages as Python-native objects and dictionaries.
- Added support for complete order lifecycle management including New, Modify, Cancel, and Trade events.
- Performed benchmarking, profiling, memory optimization, and latency analysis to improve runtime performance.
- Tech Stack: Rust, C++, PyO3, Python, Linux, Multithreading, Concurrency, Market Data Feeds, Order Book Architecture

Order Book Reconstruction from Trade and Order Messages — RMoney Nov 2025 – Present

- Built a low-latency Rust order book engine for processing exchange order and trade messages from full-day market-data files.
- Parsed trade and order messages and reconstructed the order book event by event, maintaining price-time priority.
- Maintained bid/ask sides, price levels, order quantities, market depth, and best bid/best ask in real time.
- Validated the reconstructed order book against exchange snapshots and investigated missing, duplicate, and out-of-order messages.
- Benchmarked the core processing stage on a full-day market-data file for a selected instrument token: ~3.01M filtered order/trade messages processed in 93.73ms core processing time, ~31.1ns average core latency per message, ~32.12M msg/s measured core throughput (release build; file read and token filtering measured separately).

Automatic Flood Detection System 2023 – 2024

- Designed and trained a YOLOv8-based real-time flood-level detection system using a multi-class image dataset.
- Performed model comparison, K-fold validation, latency benchmarking, error analysis, and performance evaluation.
- Optimized inference performance and analyzed deployment feasibility for real-time monitoring systems.

EDUCATION

M.Sc. in Computer Science — Dayalbagh Educational Institute Sept 2023 – May 2025

Relevant Coursework: Data Structures & Algorithms, Computer Networks, Operating Systems, Compiler Design, Data Mining, Quantum Computing, Neural Networks

B.Voc in Telematics — Dayalbagh Educational Institute Sept 2019 – May 2022

Relevant Coursework: Machine Learning, Deep Learning, NLP, Operating Systems, Python, Java